

- [MindScape Documentation](#)
 - [Table of Contents](#)
 - [1. Project Overview](#)
 - [Core Capabilities](#)
 - [2. Request Lifecycle](#)
 - [3. Core Tech Stack](#)
 - [Runtime & Framework](#)
 - [AI & Intelligence](#)
 - [Auth & Security](#)
 - [Storage](#)
 - [4. Project Structure](#)
 - [5. Data Models](#)
 - [User](#)
 - [ChatMessage](#)
 - [BehavioralData](#)
 - [6. API Endpoints Reference](#)
 - [Auth Module — /auth](#)
 - [POST /auth/register](#)
 - [POST /auth/login](#)
 - [Chat Module — /chat](#)
 - [POST /chat/](#)
 - [GET /chat/history](#)
 - [DELETE /chat/history](#)
 - [Behavior Module — /behavior](#)
 - [POST /behavior/log](#)
 - [7. Service Layer Deep-Dive](#)
 - [GroqService — services/ai_service.py](#)
 - [EmotionEngine — services/emotion_engine.py](#)
 - [AlertService — services/alert_service.py](#)
 - [8. Sentiment Detection Logic Flow](#)
 - [9. Safety Protocols & Crisis Intervention](#)
 - [Crisis Intervention Trigger Matrix](#)
 - [Safety Mode Behavior](#)
 - [Safety Event Logging](#)
 - [10. Setup & Installation](#)
 - [Prerequisites](#)
 - [Steps](#)

- [requirements.txt](#)
- [11. Environment Variables](#)
- [12. Deployment Notes](#)
 - [Render \(Configured via render.yaml\)](#)
 - [CORS Configuration \(main.py\)](#)

MindScape Documentation

Version : 1.0.0 | **Stack** : FastAPI · SQLAlchemy · Groq SDK · JWT Auth

A clinical-grade, multi-modal AI therapy and mental well-being platform — built for empathy at scale.

Table of Contents

1. [Project Overview](#)
 2. [Architecture Diagram](#)
 3. [Core Tech Stack](#)
 4. [Project Structure](#)
 5. [Data Models](#)
 6. [API Endpoints Reference](#)
 7. [Service Layer Deep-Dive](#)
 8. [Sentiment Detection Logic Flow](#)
 9. [Safety Protocols & Crisis Intervention](#)
 10. [Setup & Installation](#)
 11. [Environment Variables](#)
 12. [Deployment Notes](#)
-

1. Project Overview

Hilary AI is a premium, multi-modal AI therapist platform designed to provide empathetic, evidence-based mental health support. It combines real-time behavioral telemetry, text sentiment analysis, and large language model reasoning to deliver

personalized, clinically grounded therapy interactions rooted in **CBT (Cognitive Behavioral Therapy)** and **DBT (Dialectical Behavior Therapy)** frameworks.

Core Capabilities

Capability	Description
Conversational Therapy	LLM-driven chat using Llama 3.3-70b with structured, emotionally aware responses
Behavioral Analysis	Digital behavior heuristics derived from screen time and unlock frequency
Emotion Fusion Engine	Weighted multi-modal algorithm combining text, behavior, and sensor signals
Safety Monitoring	Proactive crisis detection with automatic alert escalation
Persistent Memory	Full conversation history with emotional state and AI insight logging
Vision/Audio (Planned)	Llama 3.2 vision + Whisper transcription for future multi-modal input

2. Request Lifecycle

```
Parse error on line 2:
...agram      autonumber      participant U a
-----^
Expecting 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW',
'SOLID_ARROW', 'DOTTED_ARROW', 'SOLID_CROSS',
'DOTTED_CROSS', got 'NL'
```

3. Core Tech Stack

Runtime & Framework

Component	Technology	Version	Rationale
Language	Python	3.9+	Async-native, rich ML ecosystem
Web Framework	FastAPI	^0.110	High performance, OpenAPI auto-docs, async support
ORM	SQLModel	^0.0.16	Unified SQLAlchemy + Pydantic — single model definition for DB and API validation
ASGI Server	Uvicorn	^0.27	Production-grade async server

AI & Intelligence

Component	Technology	Notes
AI SDK	Groq SDK	Ultra-low latency inference
Therapy Reasoning	Llama 3.3-70b	JSON Mode enabled; CBT/DBT system prompt
Vision (Future)	Llama 3.2 Vision	Facial expression / image analysis
Audio (Future)	Whisper	Voice session transcription

Auth & Security

Component	Technology	Notes
Authentication	OAuth2 Password Bearer	Industry-standard token scheme
Token Standard	JWT (HS256)	1-week expiry; signed with <code>SECRET_KEY</code>
Password Hashing	<code>passlib[bcrypt]</code>	Salted bcrypt hashing; no plaintext ever stored

Storage

Environment	Database	Notes
Development	SQLite	Zero-config local dev
Production	PostgreSQL	Configured via <code>DATABASE_URL</code> env var

4. Project Structure

```

hilary-backend/
├── main.py                # FastAPI app factory, router registration, DB
init
├── database.py            # SQLAlchemy engine, session dependency
├── models.py              # All SQLAlchemy table definitions
├── config.py              # Pydantic Settings (env var loading)
├── auth/
│   ├── router.py          # /auth endpoints
│   ├── schemas.py         # Request/response Pydantic models
│   ├── service.py         # Registration, login, JWT logic
│   └── dependencies.py    # get_current_user() dependency
├── chat/
│   ├── router.py          # /chat endpoints
│   ├── schemas.py         # ChatRequest, TherapyResponse schemas
│   └── service.py         # Orchestration: emotion → AI → DB
├── behavior/
│   ├── router.py          # /behavior endpoints
│   └── schemas.py         # BehaviorInput schema
├── services/
│   ├── ai_service.py      # GroqService: LLM calls, JSON mode
│   ├── emotion_engine.py  # EmotionEngine: multi-modal fusion
│   └── alert_service.py    # AlertService: crisis detection & escalation
├── requirements.txt
├── render.yaml            # Render deployment configuration
└── .env                  # Local secrets (never commit)

```

5. Data Models

All models use `SQLModel`, which serves as both the SQLAlchemy ORM table definition and the Pydantic validation schema.

User

```
class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    email: str = Field(unique=True, index=True)
    hashed_password: str
    is_verified: bool = Field(default=False)
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

Field	Type	Notes
id	int	Auto-increment primary key
name	str	Display name
email	str	Unique; used as login identifier
hashed_password	str	bcrypt hash — plaintext never stored
is_verified	bool	Email verification gate (default:False)
created_at	datetime	UTC timestamp of account creation

ChatMessage

```
class ChatMessage(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id")
    role: str # "user" | "assistant"
    content: str # Raw message text
    emotional_state: Optional[str] # Fused emotion label at time of
message
    detected_sentiment: Optional[str]
    intensity: Optional[int] # 1-10 distress scale
    insights: Optional[str] # AI's analytical observations (JSON
string)
    timestamp: datetime = Field(default_factory=datetime.utcnow)
```

Field	Type	Notes
role	str	"user"or "assistant"
emotional_state	str	Output of EmotionEngineat message time

Field	Type	Notes
<code>intensity</code>	<code>int</code>	1–10 scale; drives <code>AlertService</code>
<code>insights</code>	<code>str</code>	Serialized JSON from Groq JSON mode

BehavioralData

```
class BehavioralData(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id")
    screen_time_seconds: int
    unlock_count: int
    recorded_at: datetime = Field(default_factory=datetime.utcnow)
```

Field	Type	Notes
<code>screen_time_seconds</code>	<code>int</code>	Total screen-on duration in the logged period
<code>unlock_count</code>	<code>int</code>	Number of device unlock events
<code>recorded_at</code>	<code>datetime</code>	Telemetry timestamp for time-series analysis

6. API Endpoints Reference

All protected routes require the header:

```
Authorization: Bearer <access_token>
```

Auth Module — `/auth`

POST `/auth/register`

Creates a new user account.

Request Body

```
{
  "name": "Alex Chen",
  "email": "alex@example.com",
  "password": "SecureP@ssword1"
}
```

Response 201 Created

```
{
  "id": 1,
  "name": "Alex Chen",
  "email": "alex@example.com",
  "is_verified": false
}
```

Code	Meaning
------	---------

201	User created successfully
-----	---------------------------

400	Email already registered
-----	--------------------------

POST /auth/login

Authenticates a user and returns a JWT access token. Follows OAuth2 `application/x-www-form-urlencoded` convention.

Request Body (form-data)

```
username=alex@example.com
password=SecureP@ssword1
```

Response 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

Token expiry: **7 days** from issue time.

Code	Meaning
200	Login successful
401	Invalid credentials
403	Account not verified

Chat Module — /chat

POST /chat/

The core therapy loop. Accepts a user message and optional behavioral context, runs the full emotion fusion and AI pipeline, and returns a structured therapeutic response.

Request Body

```
{
  "message": "I've been feeling really overwhelmed lately and can't sleep.",
  "behavior_data": {
    "screen_time_seconds": 18000,
    "unlock_count": 47
  }
}
```

Response 200 OK

```
{
  "response": "I hear how exhausted you are, and I want you to know that what you're feeling is valid...",
  "detected_sentiment": "Anxious",
  "intensity": 7,
  "insights": [
    "User shows signs of hyperarousal consistent with anxiety.",
    "High screen time (5h) and frequent unlocks may indicate restlessness.",
    "Sleep disruption reported – CBT sleep hygiene techniques recommended."
  ],
  "emotional_state": "High Stress"
}
```

Field	Type	Notes
response	str	Therapeutic message from Llama 3.3-70b
detected_sentiment	str	Primary emotion label
intensity	int	1–10 distress scale
insights	list[str]	Analytical observations for clinician review
emotional_state	str	Fused state from <code>EmotionEngine</code>

GET /chat/history

Retrieves the authenticated user's full conversation thread in chronological order.

Response 200 OK

```
[
  {
    "id": 1,
    "role": "user",
    "content": "I've been feeling really overwhelmed lately.",
    "emotional_state": "High Stress",
    "detected_sentiment": "Anxious",
    "intensity": 7,
    "timestamp": "2025-04-14T20:31:00Z"
  },
  {
    "id": 2,
    "role": "assistant",
    "content": "I hear how exhausted you are...",
    "emotional_state": null,
    "timestamp": "2025-04-14T20:31:02Z"
  }
]
```

DELETE /chat/history

Purges all chat messages for the authenticated user. Used to begin a **New Session** .

Response 200 OK

```
{
  "detail": "Chat history cleared successfully."
}
```

```
}
```

This operation is irreversible.

Behavior Module — `/behavior`

POST `/behavior/log`

Ingests a device telemetry snapshot for the authenticated user. Called periodically by client apps through the use of Android/iOS native permissions.

Request Body

```
{
  "screen_time_seconds": 18000,
  "unlock_count": 47
}
```

Response 201 Created

```
{
  "id": 12,
  "user_id": 1,
  "screen_time_seconds": 18000,
  "unlock_count": 47,
  "recorded_at": "2025-04-14T21:00:00Z"
}
```

7. Service Layer Deep-Dive

GroqService — `services/ai_service.py`

The `GroqService` is the bridge between the application and the Groq inference API. It is configured with **JSON Mode** to guarantee structured output on every call, eliminating unpredictable free-form responses.

System Prompt Philosophy

The system prompt is clinically grounded, emphasizing:

- **Radical Empathy** : Validate before advising. Never dismiss or minimize.
- **Evidence-Based Techniques** : CBT thought records, DBT distress tolerance, grounding exercises.
- **Safety First** : Crisis language triggers an immediate pivot to safety resources.
- **Non-Diagnostic** : Hilary is a support companion, not a clinical diagnoser.

JSON Mode Contract

Every call returns exactly this schema:

```
{
  "response": "string – therapeutic message for the user",
  "detected_sentiment": "string – primary emotion (e.g., Anxious, Sad, Overwhelmed)",
  "intensity": "integer 1-10 – severity of detected distress",
  "insights": ["string – analytical observation 1", "..."]
}
```

EmotionEngine — services/emotion_engine.py

The **EmotionEngine** performs **multi-modal emotional fusion** — combining signals from three input channels into a single unified **EmotionalContext**.

Fusion Algorithm

```
Final Intensity = (W_behavior × behavior_score)
                  + (W_text    × text_sentiment_score)
                  + (W_sensor   × external_sensor_score)
```

Default weights:

Modality	Weight	Rationale
text_sentiment	0.55	Primary signal; most direct expression of state

Modality	Weight	Rationale
behavior_state	0.30	Strong indirect indicator; digital behavior is measurable
external_sensor	0.15	Supplementary; lower weight until sensor reliability is proven

Behavioral Heuristics

The engine applies rule-based heuristics to raw telemetry:

Condition	Behavioral Signal	Score
screen_time > 6h AND unlocks > 50	Severe restlessness / hyperarousal	8–10
screen_time > 4h AND unlocks > 30	Moderate stress / avoidance behavior	5–7
screen_time > 2h	Mild elevated usage	3–4
Default	Baseline / no signal	1–2

AlertService — services/alert_service.py

The `AlertService` is a passive monitoring layer that evaluates the fused `emotional_state` after every `EmotionEngine` run.

Trigger Condition

```
if emotional_state == "Critical Distress" or intensity >= CRISIS_THRESHOLD:
    self.trigger_safety_protocol(user_id, emotional_context)
```

`CRISIS_THRESHOLD` default: **9** (configurable via env var).

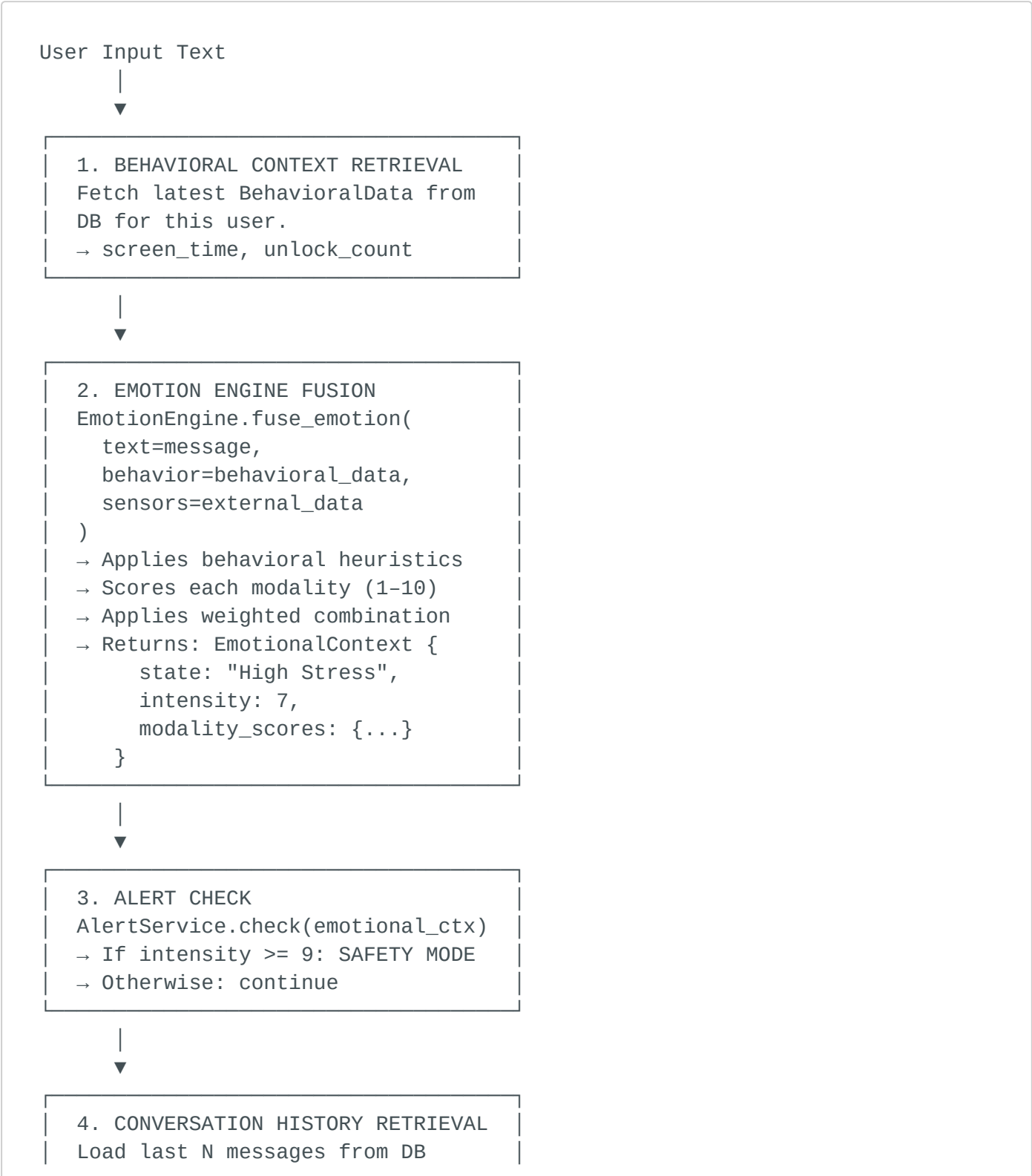
Actions on Trigger

1. Log a high-priority safety event to the database with full context.
2. Inject a crisis intervention message into the AI's next system prompt context.

- 3. (Production) Fire webhook / push notification to a human oversight system.
- 4. (Production) Append emergency resource links (e.g., 988 Suicide & Crisis Lifeline).

8. Sentiment Detection Logic Flow

This section traces the complete journey from raw user input to a saved database entry.



- Provides therapy continuity
- Prevents context amnesia



```
5. GROQ AI CALL (JSON MODE)
GroqService.get_response(
    message=text,
    emotional_context=ctx,
    history=chat_history
)
Model: Llama 3.3-70b
→ Returns structured JSON:
    {response, detected_sentiment,
     intensity, insights}
```



```
6. PERSISTENCE
Save TWO records to DB:
ChatMessage(role="user", ...)
ChatMessage(role="assistant", ...)
→ Both include emotional_state
→ Assistant record includes
    detected_sentiment + insights
```



API Response → Client

9. Safety Protocols & Crisis Intervention

Hilary operates under a **Safety-First Architecture**. The platform is designed to detect critical distress signals before the AI generates a response, ensuring intervention is woven into the therapeutic loop — not bolted on afterward.

Crisis Intervention Trigger Matrix

Signal Source	Condition	Action Level
Text Sentiment	Keywords: suicide , end it , can't go on	IMMEDIATE — Safety mode injected

Signal Source	Condition	Action Level
Fused Intensity	<code>>= 9</code> (1–10 scale)	HIGH — Crisis resources appended
Behavioral Data	<code>unlocks > 80 AND screen_time > 8h</code>	ELEVATED — Gentle check-in flagged
AI Detected Intensity	Groq returns <code>intensity >= 9</code>	HIGH — Logged as safety event

Safety Mode Behavior

When a crisis threshold is crossed, the following override is applied to the AI system prompt for that interaction:

```
SAFETY OVERRIDE ACTIVE:
- Do NOT engage with the content of the distressing statement directly yet.
- First, acknowledge the user's pain with radical, non-judgmental validation.
- Gently and clearly provide crisis support resources.
- Ask one simple, grounding question to re-establish connection.
- Resources to include: 988 (US), Crisis Text Line (Text HOME to 741741).
```

Safety Event Logging

Every crisis trigger persists a `SafetyEvent` record:

```
{
  "user_id": <int>,
  "trigger_source": "text_keyword | intensity_score | behavioral",
  "intensity_at_trigger": <int>,
  "emotional_state": "Critical Distress",
  "timestamp": <datetime>,
  "resolved": False
}
```

Production Note : A human oversight webhook should consume these events. Integrate with platforms like PagerDuty, Slack alerts, or a HIPAA-compliant case management system.

10. Setup & Installation

Prerequisites

- Python 3.9+
- A [Groq API key](#)
- `git`

Steps

```
# 1. Clone the repository
git clone https://github.com/Izumitempest/hilary-backend.git
cd hilary-backend

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate      # macOS/Linux
# .\venv\Scripts\activate    # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Create your environment file
cp .env.example .env
# → Edit .env and fill in your values (see Section 11)

# 5. Run the development server
uvicorn main:app --reload --port 8000
```

The interactive API docs will be available at:

- **Swagger UI** : <http://localhost:8000/docs>
- **ReDoc** : <http://localhost:8000/redoc>

requirements.txt

```
fastapi>=0.110.0
uvicorn[standard]>=0.27.0
sqlmodel>=0.0.16
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
```

```
python-multipart>=0.0.9
groq>=0.9.0
python-dotenv>=1.0.0
psycopg2-binary>=2.9.9      # PostgreSQL driver (production)
```

11. Environment Variables

Create a `.env` file in the project root with the following variables:

```
# — AI Configuration —————
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# — Security —————
# Generate with: python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=your-256-bit-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_DAYS=7

# — Database —————
# Leave blank for SQLite (development)
# Set to full PostgreSQL URL for production
DATABASE_URL=
# Example production value:
# DATABASE_URL=postgresql://user:password@host:5432/hilary_db

# — Safety Configuration —————
CRISIS_THRESHOLD=9

# — Application —————
DEBUG=true
APP_ENV=development    # development | staging | production
```

12. Deployment Notes

Render (Configured via `render.yaml`)

The project ships with a `render.yaml` for one-click deployment to [Render](#).

Key settings:

- **Build Command**: `pip install -r requirements.txt`
- **Start Command**: `uvicorn main:app --host 0.0.0.0 --port $PORT`

- **Database** : Provision a Render PostgreSQL instance and set `DATABASE_URL` in the Render environment variables dashboard.

CORS Configuration (`main.py`)

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://your-frontend-domain.com"], # Restrict in
production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```